

How it works — every visible behaviour traced to the line that produces it

Every visible behaviour traced to the file, function and line that produces it

docs/use-case-2/19-how-it-works-code-traces.md · rendered for review

Written 2026-09-08, for the questions raised in the client review: *which file and which function actually does this?, how does code get onto the instance?, and what exactly is on our instance now?*

Every file:line in this document was read off the source at the commit that produced it. Where a number appears it was measured on dev390988, not estimated.

1. What is on the instance

Two complete deployments of the same source. The **scoped application** is the deliverable; the **global** copy is the working fallback and is deleted only once the scoped one has passed the same audits.

	GLOBAL	SCOPED
application	—	x_2185255_command · COMMAND Analytics 0.1.0
Script Includes	9	9
UI Pages	2	2
UI Scripts (client assets)	3	3
catalog URL	/cmd_catalog.do	/x_2185255_command_cmd_catalog.do
dashboard URL	/cmd_dashboard.do	/x_2185255_command_cmd_dashboard.do

The nine Script Includes — all server-side, all in product/script-includes/ :

RECORD	WHAT IT IS RESPONSIBLE FOR
CmdMeta	the dictionary: which columns exist, their types, their choices
CmdData	every number. Counting, grouping, trends, and the ACL proof
CmdForm	which chart form a measured data shape supports, and why the others were refused
CmdAnalysis	the analyses themselves — distributions, cycles, correlations, funnels
CmdDrill	drill levels, the gates that decide whether one is offered, and the record-list link
CmdCatalog	the entry page: which subjects this viewer may analyse
CmdCeo	redrawing a Performance Analytics dashboard page
CmdPayload	assembles all of the above into one payload for one page load
CmdReport	converting an existing sys_report into the same payload

The three UI Scripts are the browser-side assets: cmd_theme (theme, first), cmd_fonts (typography), cmd_render (everything drawn).

1.1 "Why is there no CSS record?"

Because there is no CSS *record* by design. `product/ui-pages/cmd.css` is substituted into both pages at deploy time, at `deploy.py:268`, where the placeholder `@@CSS@@` in the page source is replaced with the file's contents.

Two reasons it is inlined rather than served as a stylesheet record:

1. **One request fewer.** A stylesheet record is a second round trip before the page can paint, and first paint is the number this product is judged on.
2. **One file, two surfaces.** The catalog and the dashboard share it, so they cannot drift apart visually.

The deploy refuses to inline a stylesheet containing `]]>`, `${` or `$[` (`deploy.py:263-269`) — those are sequences Jelly or XML would act on, and a stylesheet that broke the page's XML would serve a blank page rather than an error.

1.2 "Where are the UI Scripts?"

`sys_ui_script`, and their names are the part worth knowing:

global	<code>cmd_theme_2076b25a</code>	served at <code>/cmd_theme_2076b25a.jsdbx</code>
scoped	<code>cmd_theme_2076b25a</code>	served at <code>/x_2185255_command.cmd_theme_2076b25a.jsdbx</code>

The suffix is a **content hash** of the script itself. `.jsdbx` is cached hard by the platform, so a corrected script deployed under the same name keeps being served from cache — the page would be right on the instance and wrong in every browser. Putting the hash in the name means a changed script is a different URL and cannot be served stale. Superseded copies are removed by `prune_assets` (`deploy.py:319`), and only *after* the pages referencing the new ones have landed, so there is never a window where the live page points at an asset that was just deleted.

In a scoped application the field that carries the name is `script_name`; `name` is the **API Name**, which the platform computes as `<scope>.<script_name>` and which is what the URL resolves against. That distinction cost two silent outages during this build — see §5.

2. How code gets onto the instance

```
python3 product/deploy/deploy.py [--scope x_2185255_command]
```

The Python is a **build tool**. None of it is delivered: the artefact that runs on the client's instance is the update set, and it contains only platform records. The client can see the whole chain here.

read source files	product/script-includes/*.js, ui-pages/*, ui-scripts/*
validate BEFORE writing	validate_script() deploy.py:120
	validate_page() deploy.py:90
	check_asset_names() deploy.py:221
substitute	build_pages() deploy.py:240
	@@CSS@@, @@CMD_RENDER_ASSET@@ ...
select the application	Instance.use_scope() snclient.py:215
write, then READ BACK	Instance.upsert_verified() snclient.py:273
prove the classes load	verify_compiles() deploy.py:155
prove the assets are served	verify_assets_served() deploy.py:280
remove superseded assets	prune_assets()

Five things it does that a plain push does not, each of them because the alternative failed silently at least once:

It validates before it writes. `validate_page` rejects a page whose Jelly does not parse, one containing an inline `<script>` body, and one with an undefined XML entity. All three make the platform serve **zero bytes with HTTP 200 and nothing in any log**.

It refuses ES6. `validate_script` rejects `let`, arrow functions, template literals and — the one that cost a deploy round — Rhino's longer reserved-word list. `var native = ...` is a compile error that leaves the record byte-perfect and the class undefined, and the first symptom is an unrelated caller reporting "not defined".

It never trusts a status code. `upsert_verified` reads the record back and compares the stored field byte for byte against what was sent. A Table API PATCH answers 200 whether or not it stored what you gave it, and this engagement has twice reported a fix as deployed while the instance served the old record.

It proves the classes load. A readback proves bytes landed; it does not prove the script compiles. So the deploy asks the instance to construct each of the nine classes, in the engine that will run them.

It proves the assets are served. New in this build, and the check that matters most: it requests every `.jsdbx` the pages will ask for and refuses to report success unless each returns a plausible body. Readback tests storage; this tests routing, and every silent blank-page failure in this project has lived in routing.

2.1 Selecting the application, and why it is not a field

A Table API write lands in the **session's** current application, not in whatever `sys_scope` the payload names. Measured: an insert carrying `sys_scope=<app>` was created as `global.CmdProbe . use_scope` (snclient.py:215) therefore switches the session through the platform's own application picker (`PUT /api/now/ui/concoursepicker/application`), reads the application back, and **refuses to continue** unless it is the one asked for — because a deploy that silently fills the wrong scope looks exactly like one that worked.

2.2 Producing the update set the client installs

```
python3 product/deploy/package.py --version 0.1.0
```

It drives the platform's own publish rather than assembling a payload: `createUpdateSet` then `publishToUpdateSet` on `com.snc.apps.AppsAjaxProcessor`, which is what the **Publish to Update Set...** button calls, then the same

`UpdateSetExport` the **Export to XML** action runs. It counts what it produced and aborts unless the set holds the 9 Script Includes, 2 UI Pages, 3 assets and the application record.

Output: `product/dist/COMMAND-Analytics-0.1.0-update-set.xml` . Install on the target: **Retrieved Update Sets** → **Import Update Set from XML** → **Preview** → **Commit**.

3. One page load, end to end

Everything below happens on **one** request. The page does not fetch its own data: measured on this instance, an XHR from a logged-in browser session to a Scripted REST endpoint never returns, and the Table API behaves the same way from the same context. So the payload is computed server-side and embedded.

3.1 Server side — Jelly, then the Script Includes

`product/ui-pages/cmd_dashboard.xhtml`

LINE	WHAT HAPPENS
27	<code><g:evaluate></code> opens. Everything below runs on the server, in Rhino.
38–53	the drill path is parsed out of the URL — <code>field:key\ field:key</code>
108	<code>portfolio</code> parameter: a redrawn CEO page rather than a subject
113–120	<code>CmdCeo.portfolio()</code> — the CEO branch
121–123	<code>CmdReport.convert()</code> — a converted <code>sys_report</code>
124–125	<code>CmdPayload.dashboard()</code> — the ordinary subject dashboard
129–134	any failure becomes a payload carrying <code>error</code> , never a blank page
136–144	the payload is JSON-encoded and base64 -encoded
148	<code>@@@CSS@@@</code> — the stylesheet is inlined here
172–174	<code><g:requires></code> for the three client assets, by content-hashed name
152	<code><div id="cmd-data" data-b64="..."></code> — the payload, embedded

Why base64. Jelly evaluates `${...}` and `$[...]` inside both attributes and text. The payload carries field labels and record values from the instance, any of which could contain those sequences. Base64 has no `$`, no braces and no quotes, so it cannot be evaluated, cannot break the page's XML, and cannot be mangled by HTML escaping.

Why the client JavaScript is never inline. Jelly evaluates `<script>` bodies, and a CDATA-wrapped one makes the platform serve the whole page as zero bytes, HTTP 200, no error anywhere. `validate_page` enforces this at build time.

3.2 Browser side — `cmd_render.js`

BEHAVIOUR	FUNCTION	FILE:LINE
theme resolved before anything paints	(whole file)	cmd_theme.js:1
fonts, inlined as data URIs	(whole file)	cmd_fonts.js:1
entry point	<code>boot</code>	cmd_render.js:4052
decode the embedded payload	<code>payloadFrom</code>	cmd_render.js:3924
the loading skeleton	<code>paintSkeleton</code>	cmd_render.js:3958
skeleton styling	<code>.skel-block</code>	cmd.css:131
draw a dashboard	<code>renderDashboard</code>	cmd_render.js:3403
draw the catalog	<code>renderCatalog</code>	cmd_render.js:3692
one panel, header and chart	<code>buildPanel</code>	cmd_render.js:2285
the chart registry, form → renderer	<code>FORMS</code>	cmd_render.js:2104
the colour palette	<code>--c1 ...</code>	cmd.css:18
public API	<code>window.CmdRender</code>	cmd_render.js:3903

The skeleton the client saw. `paintSkeleton()` builds it from the *destination*, not a generic spinner: a dashboard skeleton carries a header, a KPI row and a chart grid because that is what a dashboard has, and a catalog skeleton carries a header and cards. It is drawn on navigation so the next page's shape is already on screen while the server computes it — which is why the transition reads as fast even though §6 shows the server is not.

There are 25 chart renderers, one per form, all in `cmd_render.js` and all named `draw*` — for example `drawTreemap` at `cmd_render.js:493`. Every one draws SVG directly. There is no charting library: nothing is fetched from a CDN at runtime, and the fonts are inlined as data URIs for the same reason. Code and fonts travel in; data does not travel out.

4. Drilldown, since it changed

A click on any value filters the page by that value.

STEP	WHERE
a mark is tagged as a drill target	<code>drillable</code> <code>cmd_render.js:788</code>
a cell meaning two values at once	<code>drillable2</code>
a bar or point meaning an interval	<code>rangeDrillable</code> , <code>periodDrillable</code>
click → navigate, hover → cross-highlight	<code>highlightLayer</code> <code>cmd_render.js:2990</code>
the URL a drill produces	<code>drillUrlSteps</code>
the path is validated server-side	<code>CmdDrill.sanitizePath</code>
a step becomes a query clause	<code>CmdDrill.stepQuery</code>
the terminal step	<code>CmdDrill.listUrl</code> → the platform's own list

22 of the 25 renderers are drillable. The drill path lives in the URL, so it is shareable and the browser's back button reverses it.

The validation is not incidental. A drill path arrives from a URL and is concatenated into an encoded query. `sanitizePath` requires each field to be a real dimension of the table and each key to contain no `^` — because

`category:software^0Rsys_idISNOTEMPTY` does not narrow a drill, it ORs in a clause matching every row, and that was confirmed live returning the whole table while still labelled VERIFIED. Range steps (`~r~lo~hi` , `~d~from~to`) are the only keys that become something other than the right-hand side of an `=` , so both endpoints must match a strict numeric or timestamp pattern and the field's declared type must agree. 25 tests cover this in `product/tests/test_drill_security.js` .

5. Two silent failures found while packaging, and what they mean for review

Both are worth reading because both passed every check that existed at the time.

The API Name is capped at 40 characters and the platform truncates rather than refusing.

`x_2185255_command.cmd_render_d5209a624137` is 41 characters. It was stored as 40. The page went on requesting all 41, so the scoped dashboard served HTTP 200 with its renderer 404ing — a blank page, nothing logged. The two smaller assets fitted, so two of three worked and it looked specific to `cmd_render` .

Then the fix for that broke global. `script_name` is the name and `name` is the API Name; a scope computes the second from the first and global computes nothing, so writing only `script_name` left `name` empty and all three *global* assets 404'd.

The lesson is in the deploy now rather than in anyone's memory: a readback proves storage and has never proved routing, so `verify_assets_served` requests every asset URL the pages will ask for and fails the deploy if one does not come back.

6. Performance, measured

Measured on dev390988, best of two runs per surface, scoped build.

SURFACE	SERVER TIME	HTML	GZIPPED
catalog	6.9 s	80 KB	21 KB
dashboard <code>incident</code>	9.5 s	164 KB	31 KB
dashboard <code>change_request</code>	6.0 s	99 KB	29 KB
CEO Portfolio 1	6.2 s	75 KB	21 KB
one drill level	4.9 s	192 KB	32 KB

Client assets, fetched once and then cached: 86 KB gzipped (theme 0.7 KB, fonts 39 KB, renderer 46 KB).

Against the stated budget:

- **Payload — passes.** First load is ≈107 KB gzipped against a 250 KB budget, and every page after that is HTML only, 21–32 KB.
- **Time — does not pass, and by a lot.** The budget is 1.2 s to first paint and 2.5 s to interactive; the server alone takes 4.9–9.5 s.

This is a real gap and it is the top of the remaining work. The cause is measured, not guessed: a page spends its time in permission-checked row scans, bounded to 6.5 s per request by `CmdData.SCAN_ALLOWANCE_MS` , and two duplications inside that budget are already identified — the ACL proof scans every row to count them and the first reduction scans the same rows again to read them, and the page opens separate reduction passes over one row set where a single pass carrying every spec would do. Closing either is worth more than the ceiling raise that currently absorbs the difference.

What the skeleton buys is perceived speed, and it is honest about it: the shape of the page is on screen immediately, but the numbers are not there until the server finishes.

7. How to check any of this yourself

```
python3 product/deploy/deploy.py --dry-run      # build and validate, write nothing
bash      product/tests/run_all.sh              # 534 offline tests
python3   product/tests/oracle_ceo.py Portfolio1 # our reading of PA against PA's own answers
python3   product/deploy/package.py            # produce the update set
```

The oracle is the one worth running before a demo: it is read-only against the client instance, and it checks our resolution of every CEO indicator against the facts table and encoded query Performance Analytics itself reports for it.